

Automated Neurosymbolic Domain Formalization for Ubuntu 25.10: A Comprehensive Research and Implementation Roadmap

1. Introduction: The Neurosymbolic Imperative in Autonomous Cyber Defense

The convergence of artificial intelligence and cybersecurity operations has precipitated a fundamental paradigm shift in how autonomous agents interact with critical infrastructure. Historically, automated remediation was the domain of static scripts—Ansible playbooks and Bash scripts—authored by human experts to address known state deviations. These deterministic systems, while reliable, are brittle; they fail when confronted with novel environment configurations or unforeseen dependencies. The advent of Large Language Models (LLMs) promised to transcend these limitations by providing agents with the semantic reasoning capabilities to interpret unstructured vulnerability reports and generate remediation strategies on the fly. However, this capabilities gain has introduced a critical vulnerability: the "probabilistic correctness" problem.¹

LLMs, operating as probabilistic token predictors, lack an inherent grounding in the invariant physics of the operating system. An LLM-driven agent does not "know" that deleting a configuration file renders a service unstartable; it merely predicts that the token sequence `rm /etc/config` is statistically associated with "resetting" an application. In high-stakes environments—such as the targeted **Ubuntu 25.10** operating system—this hallucination of system dynamics can lead to catastrophic state corruption, data loss, or the inadvertent opening of security vulnerabilities. As identified in the NDSS 2027 research roadmap, the prevailing reliance on End-to-End (E2E) generative approaches is insufficient for critical infrastructure because it lacks verifiable safety guarantees.¹

To bridge the chasm between the flexibility of LLMs and the reliability required for system administration, this report details a rigorous architectural plan to automate **Task 1a: Formalizing the 'SysAdmin' PDDL Domain**. This task proposes a **Logic-Generate-Verify (L-Gen-V)** architecture that imposes a neurosymbolic bottleneck on the agent: the opaque intent of the LLM is forced into the transparent, deterministic structure of the **Planning Domain Definition Language (PDDL)** before any executable code is synthesized.¹

However, the efficacy of a planner is bounded by the fidelity of its domain model. A manually curated `sysadmin.pddl` file is inherently subjective, prone to human error, and inevitably lags

behind the rapid evolution of the underlying operating system. This report addresses the specific challenge of formalizing the domain for **Ubuntu 25.10 "Questing Quokka,"** a release that introduces radical changes to the Linux userland, including the migration to memory-safe Rust implementations of core utilities (utils) and privilege escalation (sudo-rs).² These changes render legacy assumptions about command behavior obsolete, necessitating a system that programmatically extracts capabilities directly from the host OS.

The proposed solution eliminates subjectivity by treating the operating system as the ground truth. We detail a strategy that utilizes **osquery** for the deterministic extraction of types and predicates⁴, a **Map-Reduce** parallel processing framework for the scalable generation of action schemas from system documentation⁶, and an **Iterative Refinement Loop** guided by the **Exploration Walk (EW)** metric.¹ This approach ensures that the resulting PDDL domain is not merely a theoretical model but a verifiable digital twin of the specific Ubuntu 25.10 environment.

2. The Target Environment: Analysis of Ubuntu 25.10 "Questing Quokka"

Designing a comprehensive plan for automating domain formalization requires a nuanced understanding of the target environment. Ubuntu 25.10 represents a significant departure from previous Long Term Support (LTS) releases, introducing architectural changes that fundamentally alter the "physics" of the system administration domain. An automated extraction system must be sensitive to these shifts to avoid generating valid PDDL that corresponds to invalid system operations.

2.1 The Rust Migration: sudo-rs and utils

The most consequential change in Ubuntu 25.10 is the adoption of memory-safe replacements for critical system utilities. This "oxidization" of the userland implies that the standard behaviors documented in decades of GNU manuals may no longer hold true.

2.1.1 sudo-rs: Redefining Privilege Escalation

Ubuntu 25.10 replaces the traditional C-based sudo with **sudo-rs**, a Rust reimplementation developed by the Prossimo project and the Trifecta Tech Foundation.³ While sudo-rs aims for compatibility, it explicitly drops support for legacy features deemed insecure or obscure.

- **Behavioral Divergence:** sudo-rs does not support the preservation of the user environment via the `-E` flag in the same unrestricted manner as the legacy implementation.¹⁰ It also enforces stricter parsing of the `/etc/sudoers` file, specifically dropping support for wildcards in command arguments to prevent configuration mistakes.¹⁰

- **PDDL Implication:** A manually written PDDL action for `execute_privileged` that assumes `sudo -E` is valid would fail in execution. The automated extraction system must parse the `man sudo` output *on the specific host* to detect these constraints. If the system detects `sudo-rs`, the PDDL precondition must reflect that environment variables are not preserved, potentially requiring explicit variable passing in the action effect.
- **Security Predicates:** The move to `sudo-rs` introduces new failure modes. The automated domain generation must extract the specific exit codes and error messages associated with `sudo-rs` (e.g., handling of `NOEXEC`¹⁰) to model the `(can_escalate?user)` predicate accurately.

2.1.2 utils Coreutils: The New Atomic Actions

Ubuntu 25.10 ships with **utils coreutils**—Rust rewrites of `cp`, `ls`, `mv`, etc.—as the default providers for these essential binaries.²

- **Compatibility Gaps:** While `utils` aims for GNU compatibility, it is currently estimated at ~87.75% parity.¹³ Critical edge cases, such as specific flags in `cp` (e.g., `-Z` for SELinux context preservation) or `date` (e.g., `-r` for file reference), may be missing or behave differently.¹⁵
- **Extraction Necessity:** An LLM trained on general internet data will "hallucinate" that standard GNU flags are available. To prevent this, the extraction pipeline must query the binary itself (e.g., `cp --help`) or its local `man` page to determine the *actual* supported parameters. The PDDL action schema for `copy_file` must only include parameters that are verified to exist in the installed `utils` version.

2.2 Systemd v257: The Service Manager

Ubuntu 25.10 includes **systemd v257**, which introduces new unit file settings and deprecates others.¹⁷

- **New Capabilities:** Features like `RestartMode=debug` and `DeferReactivation=` in timer units change how services are managed.¹⁷
- **Deprecations:** Support for System V scripts and `cgroup v1` is largely removed or deprecated.¹⁸
- **PDDL Implication:** The domain formalization for `service_management` must account for these changes. The `(service_running?s)` predicate must be derived from `systemd`'s new status reporting mechanisms, and actions like `enable_service` must not assume SysV compatibility. The extraction system needs to parse `systemctl show` output to understand the dependency graph (`Before=`, `After=`, `Requires=`) which constitutes the "physics" of service sequencing.¹⁹

2.3 The Role of TPM-Backed Full Disk Encryption (FDE)

The introduction of experimental **TPM-backed FDE** in the installer³ adds a new layer of state

to the system.

- **State Dependency:** The availability of certain partitions or keys is now dependent on the Trusted Platform Module (TPM) state.
- **PDDL Modeling:** A sophisticated SysAdmin domain might need to model (tpm_unlocked?host) as a precondition for accessing encrypted volumes. While this may be out of scope for basic package management, a comprehensive plan must include mechanisms to extract encryption status via osquery or systemd-cryptsetup tools.

3. Theoretical Formalism: The SysAdmin PDDL Domain

To automate the generation of the domain, we must first define the target schema. The **SysAdmin domain** models the state and transitions of the OS. We utilize **PDDL 2.1**, prioritizing the **STRIPS** fragment for solver efficiency, but allowing for **Typing** and **Negative Preconditions**.¹

3.1 Hierarchical Type System

Subjectivity in domain modeling often stems from poor type definitions. To remove this, we define a rigid type hierarchy derived from the OS structure.

- **Base Types:** host, object.
- **Filesystem Types:** file, directory (subtypes of filesystem_object).
 - **Subtyping:** configuration_file is a critical subtype of file. It is distinguished by its functional role (configuring a service) rather than just its existence.
 - **Extraction Source:** osquery's file table, filtered by paths like /etc/.²²
- **Execution Types:** service, process.
 - **Extraction Source:** systemd_units.²³
- **Package Management Types:** package, repository.
 - **Extraction Source:** deb_packages²⁴, apt_sources.²⁵
- **Access Control Types:** user, group.
 - **Subtyping:** system_user vs. human_user based on UID ranges (e.g., UID < 1000 for system).²⁶
- **Networking Types:** port, interface, firewall_rule.
 - **Extraction Source:** listening_ports²⁷, iptables.²⁵

3.2 Predicates as State Abstractions

The core challenge in PDDL translation is the **State Abstraction Problem**.¹ The operating system state is vast and continuous (strings, timestamps, binary data). The PDDL domain requires discrete, boolean predicates.

- **Static Predicates:** These represent invariant properties of the system configuration.
 - (depends_on?s - service?p - package): Defined by package dependencies (e.g.,

- apache2.service depends on apache2 package).
- (configures?f - configuration_file?s - service): Links a config file to a service.
- (file_owned_by?f - file?u - user): Maps ownership.
- **Dynamic Predicates:** These represent the mutable state of the system.
 - (file_exists?f): True if the file is present.
 - (service_running?s): True if active_state=active.
 - (port_open?p): True if a process is bound to the port.
 - (package_installed?p): True if the package is in the deb_packages database.
 - (vulnerable?p): A derived predicate based on cross-referencing installed versions with a CVE database.

3.3 Action Schemas: Correctness-by-Construction

Action schemas define the legal transitions. The automated system must extract these from the OS capabilities.

- **install_package:**
 - *Precondition:* (not (package_installed?p)), (network_available).
 - *Effect:* (package_installed?p).
 - *Mechanism:* Uses apt-get install.
- **edit_file_parameter:**
 - *Precondition:* (file_exists?f).
 - *Effect:* (config_parameter_set?f?param?value).
 - *Mechanism:* Uses sed or echo.
- **restart_service:**
 - *Precondition:* (service_installed?s).
 - *Effect:* (service_active?s).
 - *Mechanism:* Uses systemctl restart.
- **update_package:**
 - *Precondition:* (package_installed?p), (network_available).
 - *Effect:* (not (vulnerable?p)).
- **iptables_block:**
 - *Precondition:* (package_installed iptables).
 - *Effect:* (traffic_blocked?src?dst).

4. Phase I: System Introspection (The "Map" Phase)

To remove subjectivity, the system capabilities are not manually defined but programmatically extracted from the Ubuntu 25.10 environment. This phase corresponds to the "Map" step of the parallel processing strategy.

4.1 Static Extraction via osquery

We utilize **osquery** as the primary engine for state introspection. osquery exposes operating system data as relational tables, allowing us to perform SQL queries to determine the existence and state of objects.⁴ This method provides a deterministic snapshot of the system "physics."

4.1.1 Schema Mapping Strategy

We define a mapping configuration that translates osquery schema columns into PDDL types and predicates.

PDDL Type	Osquery Table Source	Filter/Logic	PDDL Predicate Mapping
package	deb_packages ²⁴	SELECT name, version FROM deb_packages	(package_installed?p) $\rightarrow$ row exists
service	systemd_units ²³	WHERE id LIKE '%.service'	(service_running?s) $\rightarrow$ active_state='active'
user	users ²⁶	SELECT username, uid, gid FROM users	(user_exists?u) $\rightarrow$ row exists
port	listening_ports ²⁷	SELECT port, protocol FROM listening_ports	(port_open?p) $\rightarrow$ row exists
file	file ²²	Recursive query on key paths (e.g., /etc/)	(file_exists?f) $\rightarrow$ row exists
firewall_rule	iptables ²⁵	SELECT chain, policy, src_ip, dst_ip FROM iptables	(rule_exists?r) $\rightarrow$ row exists

4.1.2 Implementation Detail: The Introspector Agent

A Python module, `introspector.py`, is deployed to the target environment. It executes a predefined set of SQL queries via the `osquery` Thrift API²⁸ or the `osqueryi` shell.²⁹ The output is serialized into JSON³⁰, which serves as the "Ground Truth" for the PDDL generation.

Python

```
# Code Specification: Introspector Logic
import osquery

def extract_system_state():
    instance = osquery.SpawnInstance()
    instance.open()

    # Extract Services for PDDL Type 'service'
    # Maps systemd ActiveState to PDDL predicates
    services = instance.client.query(
        "SELECT id, active_state, sub_state FROM systemd_units WHERE id LIKE '%.service'"
    )

    # Extract Installed Packages for PDDL Type 'package'
    packages = instance.client.query("SELECT name, version FROM deb_packages")

    # Extract Users
    users = instance.client.query("SELECT username, uid FROM users")

    return {
        "services": services.response,
        "packages": packages.response,
        "users": users.response
    }
```

This programmatic approach ensures that if a service is not registered in `systemd`, it does not exist in the PDDL domain, preventing the planner from attempting to manipulate non-existent entities.

4.2 Dynamic Action Mining via Man Page Parsing

While `osquery` provides the objects (Types) and their states (Predicates), it does not define the *transitions* (Actions). To extract these, we must parse the documentation of the system utilities. Given the specific versions in Ubuntu 25.10 (`sudo-rs`, `utils`), generic internet

documentation is insufficient. We must parse the local man pages.

4.2.1 The Extraction Pipeline

1. **Documentation Rendering:** We use groff to render the roff-formatted man pages into structured ASCII text or HTML.³² This converts the dense formatting codes into a readable stream.
 - Command: `man [utility] | col -b` strips backspaces and formatting for clean text.
2. **Contextual Parsing:** We utilize **langextract**³⁴, a library designed to extract structured information from unstructured text using LLMs.
3. **Schema Generation:** The langextract library is configured with a schema that defines the components of a PDDL action: Action Name, Parameters, Preconditions (derived from flags like `--force` or requirements like "must be root"), and Effects (derived from descriptions like "removes the file").

4.2.2 Addressing sudo-rs Specifics

The extraction pipeline specifically targets `man sudo`. On Ubuntu 25.10, this points to the `sudo-rs` manual.¹¹ The LLM, analyzing this text, will identify that the `-E` flag (preserve environment) behaves differently or is discouraged compared to standard `sudo`.¹⁰

Consequently, the generated PDDL action `execute_privileged` will be constructed *without* reliance on implicit environment variable propagation, forcing the planner to be explicit about environment state. This automatic adjustment prevents the generation of plans that would fail due to the security hardening of `sudo-rs`.

5. Phase II: Parallel Synthesis (The Map-Reduce Generation)

Generating a complete `sysadmin.pddl` file for an entire OS is a massive context task that exceeds the reliable reasoning window of a single LLM call. To address this, we employ an **LLM Map-Reduce** pattern.⁶ This strategy parallelizes the generation of domain components and then merges them into a coherent whole.

5.1 The "Map" Phase: Domain Sharding

We decompose the domain generation into independent sub-tasks based on functional areas. A "Supervisor Agent" dispatches tasks to specialized "Worker Agents."

- **Worker 1 (Package Agent):**
 - *Input:* `man apt`, `man dpkg`, `osquery` package data.
 - *Task:* Generate PDDL types, predicates, and actions related to software management.
 - *Output:* A partial PDDL file containing `install_package`, `update_package`,

(package_installed), etc.

- **Worker 2 (Service Agent):**
 - *Input:* man systemctl, man journalctl, osquery service data.
 - *Task:* Generate PDDL for service lifecycle.
 - *Output:* start_service, stop_service, (service_running), (service_failed).
- **Worker 3 (Network Agent):**
 - *Input:* man iptables, man ip, osquery port data.
 - *Task:* Generate PDDL for firewall and interface management.
 - *Output:* block_ip, open_port, (port_open).
- **Worker 4 (Filesystem Agent):**
 - *Input:* man chmod, man chown, man setfacl (from utils).
 - *Task:* Generate PDDL for file manipulation.
 - *Output:* change_permissions, change_owner, (file_readable).

Each worker operates in parallel, maximizing throughput and allowing for focused context windows on specific utility documentation.

5.2 The "Reduce" Phase: Neurosymbolic Merger

The partial PDDL files produced by the workers must be synthesized. A "Merger Agent" performs the Reduce step.

- **Namespace Resolution:** The Merger identifies that Worker 1's package and Worker 2's service interact. It unifies them under a common object hierarchy.
- **Predicate Unification:** It detects that both the Service Agent and Filesystem Agent use a concept of "file existence" (e.g., for config files). It unifies these into a single global predicate (file_exists?f).
- **Consistency Check:** The Merger checks for type definition conflicts (e.g., one agent defining user as a subtype of agent and another as a subtype of object) and resolves them based on the core hierarchy defined in Section 3.1.
- **Output:** The result is a candidate sysadmin.pddl file that is syntactically complete but potentially semantically brittle.

5.3 Code Specification: Parallel Orchestrator

The following Python pseudo-code illustrates the Map-Reduce orchestration using langchain.⁶

Python

```
from concurrent.futures import ThreadPoolExecutor
```

```
from langchain.chat_models import ChatOpenAI
from langchain.prompts import PromptTemplate
```

```
# Target Utilities for Extraction
```

```
UTILITY_GROUPS = {
    "package": ["apt-get", "dpkg"],
    "service": ["systemctl"],
    "network": ["iptables", "ip"],
    "file": ["chmod", "chown", "acl"]
}
```

```
def map_worker(group_name, utilities):
```

```
    """
```

```
    MAP STEP: Generates a partial PDDL domain for a specific utility group.
```

```
    """
```

```
    # 1. Fetch Man Pages (System Introspection)
```

```
    docs = [fetch_man_page(u) for u in utilities]
```

```
    # 2. Extract Action Schemas via LLM
```

```
    action_schemas =
```

```
    for doc in docs:
```

```
        schema = llm_extract_action(doc) # Uses langextract
```

```
        action_schemas.append(schema)
```

```
    # 3. Return Partial Domain
```

```
    return generate_partial_pddl(group_name, action_schemas)
```

```
def reduce_merger(partial_domains):
```

```
    """
```

```
    REDUCE STEP: Merges partial domains into a unified sysadmin.pddl.
```

```
    """
```

```
    # 1. Unify Types
```

```
    unified_types = merge_types([d.types for d in partial_domains])
```

```
    # 2. Unify Predicates
```

```
    unified_predicates = merge_predicates([d.predicates for d in partial_domains])
```

```
    # 3. Concatenate Actions
```

```
    unified_actions = [a for d in partial_domains for a in d.actions]
```

```
    # 4. Validate Syntax (using VAL)
```

```
    candidate_domain = construct_domain(unified_types, unified_predicates, unified_actions)
```

```
    if validate_pddl(candidate_domain):
```

```

return candidate_domain
else:
return repair_pddl(candidate_domain) # Self-correction loop

```

6. Phase III: Iterative Refinement via Exploration Walks

A statically generated PDDL domain is rarely perfect. Small errors—such as missing a precondition that apt requires sudo—can render the entire domain unusable for planning. To address this, we implement the **Domain Refinement** technique utilizing the **Exploration Walk (EW)** metric, as detailed in the paper "Leveraging Environment Interaction for Automated PDDL Translation and Planning with Large Language Models".¹

6.1 The Refinement Algorithm

The refinement process is an automated, closed-loop search through the space of possible domain definitions. It does not rely on human feedback; instead, it uses the environment itself as the critic.

1. **Mental Model Construction:** The candidate sysadmin.pddl generated in Phase 2 serves as the initial "mental model."
2. **Exploration Walk (Trace Generation):** The system uses a planner (Fast Downward³⁵) to generate random, valid sequences of actions (π) based on the *mental model*. For example: (update_package pkg1) -> (restart_service srv1).
3. **Environment Interaction (Grounding):** These sequences are executed in a sandboxed **Ubuntu 25.10 Docker container**. This is the "Reality Check."
4. **Discrepancy Observation:** The system compares the expected outcome (from the PDDL effects) with the actual outcome (exit codes, state changes).
 - *Example Discrepancy:* The PDDL predicts restart_service succeeds. The container returns exit code 1 because the configuration file was missing (a constraint not modeled in the PDDL).
5. **Feedback & Update:** This discrepancy ("Action failed: missing config file") is fed back to the LLM. The LLM updates the PDDL action restart_service to include a new precondition (file_exists?config).

6.2 The Exploration Walk (EW) Metric

To quantitatively guide this search, we calculate the EW score.¹ This metric measures the alignment between the PDDL model and the real world.

$$EW(d) = \frac{1}{N \cdot T_{\max}} \sum_{j=1}^N \sum_{T=1}^{T_{\max}} \mathbb{E}_q \sim \mathbb{P}_{\{d, p_j, T\}} [E_{\text{env}}(q)]$$

Where:

- d is the candidate domain.
- q is a random action sequence of length T sampled from domain d .
- $E_{env}(q)$ is the executability function: 1 if the sequence executes successfully in the Ubuntu 25.10 container, 0 otherwise.

A high EW score indicates that the "mental model" accurately predicts what is possible in the real world. The refinement loop continues until the EW score exceeds a defined threshold (e.g., 0.9).

6.3 Implementation of the Refiner Module

The refiner module (refiner.py) integrates the planner and the container environment.

Python

```
import docker
from fast_downward_wrapper import PlanGenerator

class DomainRefiner:
    def __init__(self, pddl_domain):
        self.domain = pddl_domain
        self.client = docker.from_env()
        # Target Ubuntu 25.10 specifically for valid grounding
        self.container = self.client.containers.run("ubuntu:25.10", detach=True, tty=True)

    def calculate_ew_score(self, num_walks=10, depth=5):
        successes = 0
        total_steps = 0

        for _ in range(num_walks):
            # Generate random walk from Mental Model (PDDL)
            plan = PlanGenerator(self.domain).sample_random_walk(depth)

            for action in plan:
                total_steps += 1
                # Concretize to Bash (e.g., apt-get install...)
                cmd = self.concretize(action)

            # Execute in Reality (Container)
```

```
exec_result = self.container.exec_run(cmd)
```

```
if exec_result.exit_code == 0:
```

```
    successes += 1
```

```
else:
```

```
    # Discrepancy found! Stop this walk.
```

```
    self.log_feedback(action, exec_result.output)
```

```
    break
```

```
return successes / total_steps
```

```
def refine(self):
```

```
    """
```

```
    Iterative Tree Search for Domain Optimization
```

```
    """
```

```
    current_score = 0
```

```
    while current_score < 0.9:
```

```
        current_score = self.calculate_ew_score()
```

```
        if current_score < 0.9:
```

```
            # LLM updates PDDL based on failure logs
```

```
            self.domain = self.llm_update_domain(self.feedback_logs)
```

7. Implications for Reproducibility and Security

7.1 Deterministic Grounding

By anchoring the PDDL domain in osquery tables, we achieve a level of determinism previously unattainable in manual modeling. The domain is not an expert's *opinion* of the system; it is a *projection* of the system's actual state database. This directly addresses the "Subjectivity" constraint of the user request.

7.2 Safety in sudo-rs Environments

The automated extraction of the sudo-rs action schema ensures that the agent respects the specific security boundaries of Ubuntu 25.10. A manual domain might assume it can run `sudo -E python script.py` to preserve environment variables. The automated pipeline, detecting sudo-rs documentation, will recognize this is unsupported or risky, potentially restructuring the plan to pass variables explicitly. This prevents "silent failures" where a script runs but lacks the necessary context to function.

7.3 Scalability via Parallelism

The Map-Reduce architecture allows the system to scale to cover the vast breadth of Linux utilities. As new tools are added to the OS (e.g., a new container runtime), only a new "Worker Agent" needs to be instantiated. The "Merger Agent" automatically integrates the new capabilities into the sysadmin.pddl, maintaining the domain's comprehensiveness without manual refactoring.

8. Conclusion

This report presents a robust, executable roadmap for automating the formalization of the SysAdmin PDDL domain for Ubuntu 25.10. By integrating **System Introspection (osquery)**, **Parallel Synthesis (Map-Reduce)**, and **Iterative Validation (Exploration Walk)**, the proposed architecture effectively removes human subjectivity from the loop. It produces a domain model that is not only syntactically correct but semantically grounded in the specific "physics" of the target operating system.

The shift to **Rust-based core utilities** in Ubuntu 25.10 is not merely a detail but a driving factor in the design of the extraction pipeline. The dynamic parsing of local man pages ensures that the agent's internal model aligns perfectly with the actual binaries installed, preventing the "hallucination of capability" that plagues generic LLM agents. This work lays the foundation for the verifiable, autonomous remediation agents envisioned in the NDSS 2027 roadmap, moving us closer to a future where self-healing systems are both intelligent and mathematically provable.

9. Summary of Data Sources & Schema Mappings

Component	Source of Truth	Extraction Method	PDDL Mapping
Object Types	Ubuntu 25.10 Live System	osquery SQL queries (tables: deb_packages, systemd_units, users) ⁴	(:types package service user...)
Predicates	System State Snapshots	Mapping osquery schema columns to boolean flags (e.g., active_state -> service_running) ²⁷	(:predicates (service_running?s))

Action Syntax	utils & sudo-rs Binaries	`man [cmd]	groff&[cmd] --help` passed to LLM ²
Refinement	Execution Feedback	Exploration Walk (EW) metric running in Dockerized Ubuntu 25.10 ¹	Domain updates based on execution success rate
Planning	PDDL 2.1 Standard	Fast Downward planner & VAL validator ³⁵	Plan verification & synthesis

Works cited

1. NDSS 2027 Paper Planning.pdf
2. Canonical releases Ubuntu 25.10 Questing Quokka, accessed January 5, 2026, <https://canonical.com/blog/canonical-releases-ubuntu-25-10-questing-quokka>
3. What's new in security for Ubuntu 25.10?, accessed January 5, 2026, <https://ubuntu.com/blog/ubuntu-25-10-security-updates>
4. How to Monitor Linux Server Security with Osquery - GeeksforGeeks, accessed January 5, 2026, <https://www.geeksforgeeks.org/linux-unix/how-to-monitor-linux-server-security-with-osquery/>
5. Query your systems like a database with OSQuery | by Safeer CM | The Cloud Bulletin, accessed January 5, 2026, <https://medium.com/cloudbulletin/query-your-systems-like-a-database-with-osquery-55cee13f696d>
6. LLM×MapReduce-V3: Enabling Interactive In-Depth Survey Generation through a MCP-Driven Hierarchically Modular Agent System - arXiv, accessed January 5, 2026, <https://arxiv.org/html/2510.10890v1>
7. LLM×MapReduce: Simplified Long-Sequence Processing using Large Language Models - ACL Anthology, accessed January 5, 2026, <https://aclanthology.org/2025.acl-long.1341.pdf>
8. Adopting sudo-rs By Default in Ubuntu 25.10 - Project Discussion, accessed January 5, 2026, <https://discourse.ubuntu.com/t/adopting-sudo-rs-by-default-in-ubuntu-25-10/60583>
9. sudo-rs Headed to Ubuntu - Prossimo - Memory Safety, accessed January 5, 2026, <https://www.memorysafety.org/blog/sudo-rs-headed-to-ubuntu/>
10. trifactatechfoundation/sudo-rs: A memory safe implementation of sudo and su. - GitHub, accessed January 5, 2026, <https://github.com/trifactatechfoundation/sudo-rs>

11. sudoers-rs(5) - Arch manual pages, accessed January 5, 2026,
<https://man.archlinux.org/man/extra/sudo-rs/sudoers-rs.5.en>
12. 10 Biggest Ubuntu Changes in 2025, accessed January 5, 2026,
<https://www.omgubuntu.co.uk/2026/01/ubuntu-2025-year-in-review>
13. Rust Coreutils 0.5.0 Release: 87.75% compatibility with GNU Coreutils | Hacker News, accessed January 5, 2026,
<https://news.ycombinator.com/item?id=46264329>
14. Rust Coreutils 0.5.0: 87.75% compatibility with GNU Coreutils : r/linux - Reddit, accessed January 5, 2026,
https://www.reddit.com/r/linux/comments/1pmltla/rust_coreutils_050_8775_compatibility_with_gnu/
15. Bug in Rust coreutils rewrite breaks automatic updates in Ubuntu 25.10 - Reddit, accessed January 5, 2026,
https://www.reddit.com/r/programming/comments/1oeiulp/bug_in_rust_coreutils_rewrite_breaks_automatic/
16. What do you think about replacing gnu core-utils with utils by default in nixos?, accessed January 5, 2026,
<https://discourse.nixos.org/t/what-do-you-think-about-replacing-gnu-core-utils-with-utils-by-default-in-nixos/61797>
17. Systemd 257 Released, Here's What's New - Linuxiac, accessed January 5, 2026,
<https://linuxiac.com/systemd-257-linux-service-manager-released/>
18. systemd 257 Debuts With systemd-keyutil & systemd-sbnsign Tools, Other Improvements - Phoronix, accessed January 5, 2026,
<https://www.phoronix.com/news/systemd-257-Released>
19. Systemd dependencies and boot order - linux - Server Fault, accessed January 5, 2026,
<https://serverfault.com/questions/482730/systemd-dependencies-and-boot-order>
20. Ubuntu 25.10: What's New, What's Changed & Should You Upgrade?, accessed January 5, 2026,
<https://www.omgubuntu.co.uk/2025/10/ubuntu-25-10-new-features>
21. (PDF) PDDL - The Planning Domain Definition Language - ResearchGate, accessed January 5, 2026,
https://www.researchgate.net/publication/2278933_PDDL_-_The_Planning_Domain_Definition_Language
22. The file table: osquery's secret weapon - 1Password, accessed January 5, 2026,
<https://1password.com/blog/the-file-table-osquerys-secret-weapon>
23. "systemd_units" in osquery | Fleet documentation, accessed January 5, 2026,
https://fleetdm.com/tables/systemd_units
24. osquery/specs/linux/deb_packages.table at master - GitHub, accessed January 5, 2026,
https://github.com/osquery/osquery/blob/master/specs/linux/deb_packages.table
25. linux_osquery - GitHub Gist, accessed January 5, 2026,
<https://gist.github.com/githubfoam/0babb95da5845b8d4ee41f5711de637a>
26. What is Osquery? Find Out How it Works & How to Use it! - Uptycs, accessed

January 5, 2026,

<https://www.uptycs.com/blog/threat-research-report-team/osquery-guide>

27. osquery/specs/listening_ports.table at master - GitHub, accessed January 5, 2026, https://github.com/osquery/osquery/blob/master/specs/listening_ports.table
28. SDK and Extensions - osquery - Read the Docs, accessed January 5, 2026, <https://osquery.readthedocs.io/en/stable/development/osquery-sdk/>
29. osqueryi (shell) - osquery - Read the Docs, accessed January 5, 2026, <https://osquery.readthedocs.io/en/stable/introduction/using-osqueryi/>
30. Ultimate Guide to JSON Parsing in Python - Scrapfly, accessed January 5, 2026, <https://scrapfly.io/blog/posts/how-to-use-python-to-parse-json>
31. How can I parse (read) and use JSON in Python? - Stack Overflow, accessed January 5, 2026, <https://stackoverflow.com/questions/7771011/how-can-i-parse-read-and-use-json-in-python>
32. Man page of GROFF, accessed January 5, 2026, <https://sites.uclouvain.be/SystInfo/manpages/man1/groff.1.html>
33. The quest to collect all software man pages - Emad Elsaid, accessed January 5, 2026, <https://www.emadelsaid.com/The%20quest%20to%20collect%20all%20software%20man%20pages/>
34. google/langextract: A Python library for extracting structured information from unstructured text using LLMs with precise source grounding and interactive visualization. - GitHub, accessed January 5, 2026, <https://github.com/google/langextract>
35. Integration of the planning system Fast Downward with the unified-planning framework. - GitHub, accessed January 5, 2026, <https://github.com/aiplan4eu/up-fast-downward>
36. patrikhaslum/INVAL: The INVAL plan validator, and other PDDL tools. - GitHub, accessed January 5, 2026, <https://github.com/patrikhaslum/INVAL>